



**UNIVERSIDAD
DE GRANADA**

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

Aplicación Del Procesamiento Del Lenguaje Natural Cuántico

Autor

Daniel Terés Martínez

Directores

Manuel Pegalajar Cuéllar

Serafín Moral García



**ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN**

—
4 de marzo de 2026

Aplicación Del Procesamiento Del Lenguaje Natural Cuántico

Daniel Terés Martínez

Palabras clave: palabra_clave1, palabra_clave2, palabra_clave3,

Resumen

Poner aquí el resumen.

Yo, **Daniel Terés Martínez**, alumno de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 41542052G, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Daniel Terés Martínez

Granada 4 de marzo de 2026.

D. **Manuel Pegalajar Cuéllar**, Profesor del Área de XXXX del Departamento Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

D. **Serafín Moral García**, Profesor del Área de XXXX del Departamento Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***Aplicación Del Procesamiento Del Lenguaje Natural Cuántico***, ha sido realizado bajo su supervisión por **Daniel Terés Martínez**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada 4 de marzo de 2026.

Los directores:

Manuel Pegalajar Cuéllar Serafín Moral García

Agradecimientos

Poner aquí agradecimientos...

Índice general

1. Introducción	1
1.1. Objetivos	1
1.2. Motivación	1
2. Números complejos	3
2.1. ¿Qué es un número complejo?	3
2.2. Representación binómica	3
2.3. Representación geométrica en $\mathbb{R}^2$	4
2.4. Representación forma polar	5
2.4.1. Operaciones	6
2.5. Representación en forma exponencial (Euler)	7
3. Notación Dirac	9
3.1. Notación bra-ket	9
4. ¿Qué es un Qubit?	11
4.1. ¿Cómo expresarlo con notación Dirac?	11
4.2. Esfera de Bloch	12
4.3. Puertas cuánticas	14
5. Procesamiento Lenguaje Natural (PLN)	17
5.1. ¿Qué es el procesamiento de lenguaje natural?	17
5.2. Fases del PLN	17
5.2.1. Preprocesamiento y Tokenización	18
5.2.2. Análisis sintáctico	19
5.2.3. Análisis semántico	19
5.2.4. Integración del discurso	20
5.2.5. Análisis pragmático	20
6. Procesamiento del Lenguaje Natural Cuántico (PLNC)	21
6.1. Motivación para el uso de PLNC	21
6.2. Codificación de información clásica en estados cuánticos	22
6.2.1. Técnicas de codificación de datos clásicos a estados cuánticos	23

6.3. Circuitos Variacionales Cuánticos (CVC) como modelos de aprendizaje	25
7. Representación Vectorial de Palabras (Word Embeddings)	27
7.1. Introducción al concepto de <i>Embedding</i>	27
7.1.1. Embeddings de palabras (<i>Word Embeddings</i>)	28
7.2. Mecanismos geométricos de los embeddings	30
7.2.1. Métricas de Distancia	30
7.2.2. Información Latente	32
7.3. Paralelismo entre Espacio Semántico y Espacio de Hilbert . .	34
7.3.1. Codificación multidimensional: Forma y Contenido . .	34
7.3.2. Afinidad Cuántica, Superposiciones y Fases	35
8. Implementación Clásica: Algoritmo Word2Vec	37
8.1. Arquitectura del modelo	37
8.1.1. Modelos de entrenamiento: CBOW y Skip-gram	39
8.2. Función de optimización	43
9. Implementación Cuántica: Algoritmo QWord2Vec	47
9.1. Adaptación del algoritmo al entorno cuántico	47
9.2. Diseño del Circuito Cuántico Parametrizado	47
9.3. Cálculo de similitud en hardware cuántico (Test de Swap / Fidelidad)	50
9.4. Definición de la función de coste cuántica	50
9.5. Estrategia de optimización híbrida (Clásica-Cuántica)	50
10. Experimentación y Análisis de Resultados	51
10.1. Metodología experimental	51
10.1.1. Dataset y preprocesamiento (corpus)	51
10.1.2. Entorno de simulación	51
10.2. Entrenamiento	51
10.2.1. Gráficas de pérdida: Word2Vec vs QWord2Vec	51
10.3. Evaluación de calidad de los embeddings	51
10.3.1. Pruebas de analogía y similitud	51
10.4. Comparativa de recursos computacionales y tiempos	51
Bibliografía	53
Glosario	57

Capítulo 1

Introducción

1.1. Objetivos

1.2. Motivación

Capítulo 2

Números complejos

2.1. ¿Qué es un número complejo?

Un número complejo (representado por el símbolo $\mathbb{C}$) es la combinación de números reales e imaginarios. La parte real se puede expresar por un número entero o sus decimales, la parte imaginaria es aquella cuyo cuadrado es negativo.

Los números complejos sirven para dar sentido a la raíz cuadrada de números negativos. Esto permite que ecuaciones que tengan como resultado una raíz cuadrada negativa, puedan ser resueltas. Con esto se abre la puerta a un gran mundo en el que todas las operaciones (excepto dividir entre 0), son posibles.

Las características principales de los números complejos con las siguientes:

- Tiene distintas formas de expresarse.
- Dos números complejos se consideran iguales cuando tienen mismo componente real e imaginario.
- $\mathbb{C}$ conforma un espacio vectorial de dos dimensiones.
- Los números complejos no pueden mantener un orden. Es decir no se puede encontrar una propiedad $>$ o $<$ que un número para todos los números complejos.

2.2. Representación binómica

Consideremos la unidad imaginaria $i = \sqrt{-1}$ la cual representamos en el punto $(0, 1)$ del plano. Con esto, se puede situar los números $2i, 3i, \dots, -i, -2i, \dots$ en el eje vertical, a estos números que tienen forma bi , los llamaremos imaginarios. Entonces cualquier punto en el plano (a, b) donde $a, b, \in \mathbb{R}$, se puede

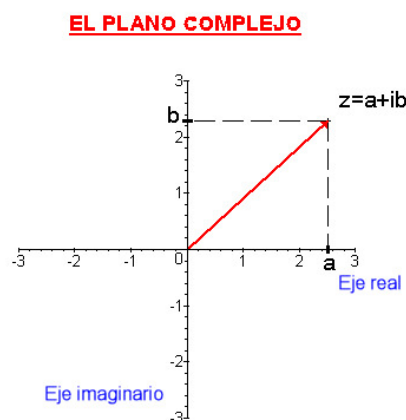


Figura 2.1: Plano sobre como representar número complejo

expresar como $(a, 0) + (0, b)$, siendo la suma de un número real y un número imaginario. Su representación es la siguiente:

$$z = a + bi, \quad a, b \in \mathbb{R}, \quad i = \sqrt{-1}$$

Con esta representación, a es la parte real y b la parte imaginaria. Se puede representar en el plano mediante el punto $Z(a, b)$, a esto se le conoce como **afijo**.

- **Conjugado:** Se define de la forma $\bar{z} = a - ib$.
- **Inverso:** Se define como $z^{-1} = \frac{a}{a^2+b^2} - \frac{b}{a^2+b^2}i$.

Los números complejos se pueden sumar, restar, multiplicar y dividir siguiendo las reglas de las operaciones con números reales.

2.3. Representación geométrica en $\mathbb{R}^2$

Para representar gráficamente un número complejo, debemos dibujarlo en el plano complejo. Está formado por un **eje real** y **eje imaginario**, en el eje real se representa la parte real mientras que en el eje imaginario la parte imaginaria.

Basándonos en la definición del apartado anterior sobre el inverso y conjugado de un número complejo, al representarlo de forma gráfica queda de la siguiente manera:

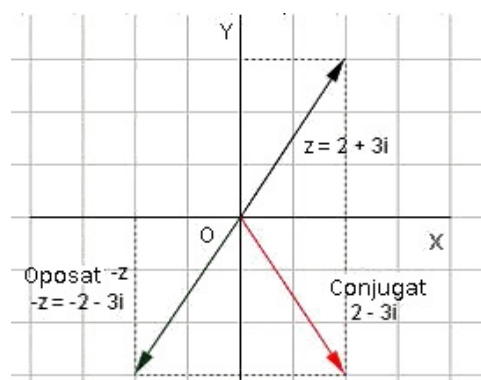


Figura 2.2: Representación en plano sobre conjugado e inverso

2.4. Representación forma polar

Esta representación sirve para destacar los atributos gráficos que tienen los números complejos.

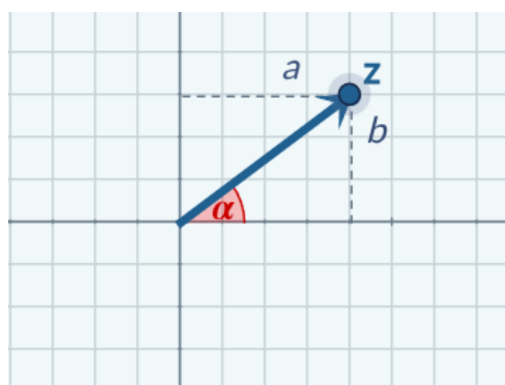


Figura 2.3: Gráfica de la forma polar de un número complejo

- **Módulo (r):** Distancia del afijo al origen de coordenadas.
- **Argumento ($\arg(z)$):** Ángulo que forma el segmento $\overline{OZ}$ con el semieje positivo medido en el sentido contrario de las agujas del reloj entre 0° y 360° .

Con el módulo y el argumento se puede expresar el número complejo z en **forma polar** de la siguiente forma:

$$r_\alpha \quad \text{con } r = |z| \text{ y } \alpha = \arg(z)$$

En base a la forma polar, se puede expresar **z** en **forma trigonométrica** de la siguiente manera:

$$z = r(\cos \alpha + i \operatorname{sen} \alpha),$$

Esto nos puede servir para pasar de forma polar a forma binómica.

2.4.1. Operaciones

Con esta representación, se permite realizar la multiplicación y división de una manera sencilla. Para ello hay que **expresar los números complejos en forma trigonométrica**.

- **Producto:** $r \cdot r' (\cos(\alpha + \beta) + i \operatorname{sen}(\alpha + \beta))$
Se observa que el módulo del número complejo es el resultante del producto de los módulos y que el argumento es la suma de los argumentos.
- **Cociente:** $\frac{r}{r'} (\cos(\alpha - \beta) + i \operatorname{sen}(\alpha - \beta))$ El resultado del cociente es el cociente de los módulos y la diferencia de los argumentos

Por lo que en **forma polar** quedaría de la siguiente manera:

- **Producto:** $r_\alpha \cdot r'_\beta = (r \cdot r')_{\alpha+\beta}$
- **Cociente:** $\frac{r_\alpha}{r'_\beta} = \left(\frac{r}{r'}\right)_{\alpha-\beta}$

Para calcular **potencia** con exponente natural **n**, en forma polar, hay que elevar el módulo a **n** y multiplicar el argumento por **n**.

$$(r_\alpha)^n = (r^n)_{n \cdot \alpha}$$

La **raíz n-esima** de un número complejo en forma polar, es otro número complejo m_β que debe de cumplir:

$$(m_\beta)^n = r_\alpha$$

$$m = \sqrt[n]{r} \quad \text{y} \quad \beta = \frac{\alpha}{n} + \frac{360^\circ * k}{n}$$

Todo número complejo tiene n raíces n-ésimas. Todos los afijos están situados sobre una circunferencia y son los vértices de un polígono regular de n lados.

Un ejemplo de forma gráfica calculando la raíz n-esima con lo explicado anterioremente, es el siguiente:

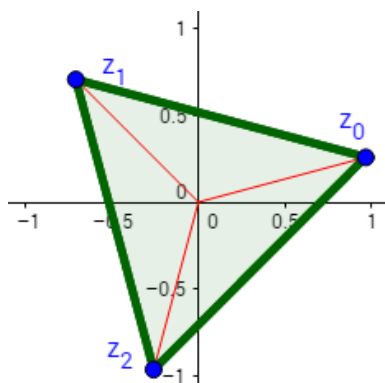


Figura 2.4: Raíz 3-esima del número complejo $z = i + 1$

2.5. Representación en forma exponencial (Euler)

El número de Euler (e) es un número constante e irracional. Se puede definir a través de la Serie de Taylor. Este número es la base para resolver ecuaciones relacionadas con el crecimiento exponencial, así como en logaritmos neperianos los cuales sirven para manejar números grandes de una manera simplificada para poder realizar cálculos.

La representación del número complejo con el número de Euler, viene dado porque Leonhard Euler aplicó la Serie de Taylor poniéndole la i del número imaginario, luego simplificó teniendo en cuenta que $i^2 = -1$. Al aplicar lo comentado, le quedó lo siguiente:

$$e^{ix} = \left(1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots\right) + i \left(x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots\right)$$

Se dio cuenta que los dos paréntesis de la expresión anterior, es la Serie de Taylor aplicada al **cos** y **sen**.

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots$$

$$\text{sen } x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

Con el seno y coseno, se simplifica en:

$$e^{ix} = \cos x + i \text{sen } x$$

Este resultado es la representación del número complejo mediante el número de Euler. Además, se le conoce como la **fórmula de Euler**.

Capítulo 3

Notación Dirac

La notación de Dirac nos da una manera de expresar estados en un espacio vectorial sin atarnos a un sistema de coordenadas específico, pero que hace muy sencillo elegir una base concreta y cambiar de una base a otra.

3.1. Notación bra-ket

En un **ket** los valores están representados en un vector columna mientras que en un **bra** los valores están representados como un vector fila.

- **ket:** Representa un vector con valores complejos, se denota como: $\vec{a} = |a\rangle$
- **bra:** Representa el conjugado del vector complejo ket, se denota como: $\vec{a}^* = \langle a|$

Para **realizar la transformación** entre un **bra** y un **ket**, hay que realizar el conjugado de la transpuesta.

la base canónica en $\mathbb{C}^2$ viene representado en vector base como $e_1 = (1, 0)$ y $e_2 = (0, 1)$. Por lo que en notación ket quedaría como $|0\rangle = (1, 0)$ y $|1\rangle = (0, 1)$.

- **Producto interno:** $\langle a|b\rangle = \begin{pmatrix} a_1^* & a_2^* \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = a_1^* b_1 + a_2^* b_2$
- **Producto externo:** $\mathbf{A} = \sum_{i,j=1}^N A_{ij} |e_i\rangle\langle e_j| = \sum_{i,j=1}^N |e_i\rangle A_{ij} \langle e_j|$

Capítulo 4

¿Qué es un Qubit?

Un Qubit es el análogo cuántico al bit clásico. Mientras el bit tiene **dos posibles estados**, el 0 y el 1, el Qubit se representa como un vector en el espacio vectorial $\mathbb{C}^2$. La base del espacio más utilizada, se denomina base computacional y se corresponde con la base canónica de $\mathbb{C}^2$, que en notación de Dirac, son los vectores $|0\rangle$ y $|1\rangle$:

$$\begin{aligned} |0\rangle &= \begin{bmatrix} 1 \\ 0 \end{bmatrix} \in \mathbb{C}^2 \\ |1\rangle &= \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{aligned}$$

Un **estado cuántico** se corresponde con la descripción matemática de vector que representa al Qubit.

4.1. ¿Cómo expresarlo con notación Dirac?

Un qubit $|\Psi\rangle$ puede estar en una superposición de $|0\rangle$ y $|1\rangle$, lo que significa que es una combinación lineal entre los vectores de la base. Esto se representa como:

$$|\Psi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle$$

Donde α_0 y α_1 son las **amplitudes de los estados**, además son también números complejos. Por lo tanto, mientras que un bit puede solo tener 1 entre 2 valores (0 y 1), un qubit puede tener uno entre finitos posibles estados. Además, se tiene que cumplir que la norma del vector sea unitaria ($|\alpha_0|^2 + |\alpha_1|^2 = 1$).

Hay que tener en cuenta que no existe una forma directa de determinar los valores de α_1 y α_2 , ya que el estado cuántico no es directamente observable.

Antes de realizar una medición, el qubit se encuentra en una superposición de los estados $|0\rangle$ y $|1\rangle$, con ciertas probabilidades asociadas a cada uno.

Para conocer el resultado de una computación cuántica, es necesario medir los qubits. Sin embargo, este proceso de medición implica interactuar con el qubit, lo que perturba su estado y hace que este deje de estar en superposición.

Al medirlo, el qubit colapsa de forma definitiva a uno de los dos estados posibles ($|0\rangle$ o $|1\rangle$), perdiéndose la información sobre las amplitudes originales que este tenía.

La probabilidad de que el qubit colapse en un estado concreto se determina mediante la regla de Born, la cual establece que dicha probabilidad es igual al módulo al cuadrado de la amplitud correspondiente a ese estado.

La superposición, tiene distintas representaciones que son:

- **Binómica:** $x_1 + iy_1 |0\rangle + x_2 + iy_2 |1\rangle$
- **Exponencial:** $r_0 e^{i\phi_0} |0\rangle + r_1 e^{i\phi_1} |1\rangle$
- **Senos y cosenos:** $|\Psi\rangle = \cos \frac{\phi}{2} |0\rangle + e^{i\theta} \sin \frac{\phi}{2} |1\rangle$
- **Fase global:** Factor común $e^{i\gamma}$ que multiplica a todo el estado $|\psi\rangle$. Este no cambia ninguna probabilidad ni resultado físico.
- **Fase relativa:** Es la diferencia entre las amplitudes α y β . Esto influye a las inferencias y los resultados cuando se miden en otras bases (X, Y) .

4.2. Esfera de Bloch

La esfera de Bloch es una representación visual del estado de un qubit. Cada punto sobre su superficie corresponde a una posible superposición de los estados base $|0\rangle$ y $|1\rangle$. Los polos superior e inferior representan los estados $|0\rangle$ y $|1\rangle$, respectivamente, y el estado del qubit se indica con una flecha que apunta hacia el punto que lo describe.

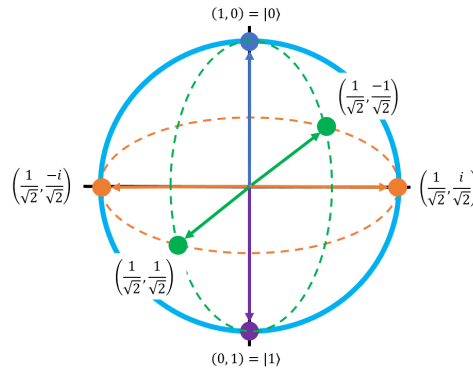


Figura 4.1: Imagen obtenida de <https://stem.mitre.org/quantum/quantum-concepts/bloch-sphere.html>

Importante: la notación $\left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$ se corresponde con el vector estado $\begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$ que a su vez significa:

$$\begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix} = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

Los seis puntos principales de la esfera de Bloch, son los que se ve en la Figura 4.1. A estas coordenadas principales, se le pueden asignar etiquetas.

- **Base de Hadamard:** También se le conoce como eje X de Pauli. El punto $\left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)$ se corresponde con $|+\rangle$. A su vez, el punto opuesto $\left(\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right)$ se corresponde con $|-\rangle$
- **Base circular:** También se le conoce como eje Y de Pauli. El punto $\left(\frac{1}{\sqrt{2}}, \frac{i}{\sqrt{2}}\right)$ se corresponde con $|+i\rangle$. A su vez, el punto opuesto $\left(\frac{1}{\sqrt{2}}, -\frac{i}{\sqrt{2}}\right)$ se corresponde con $|-i\rangle$

Con dicha asignación de etiquetas, queda la siguiente esfera:

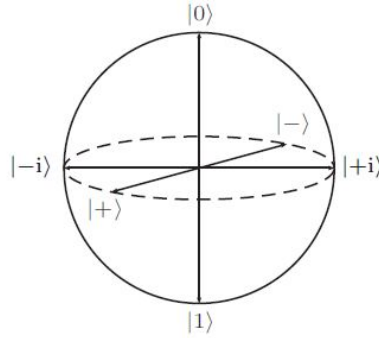


Figura 4.2: Imagen obtenida de https://en.wikipedia.org/wiki/Bloch_sphere

4.3. Puertas cuánticas

Al igual que para manipular los valores de los bits existen puertas lógicas clásicas como OR, AND, NOT, etc. Los ordenadores cuánticos, manipulan los valores de los qubits usando puertas cuánticas. Es decir, cualquier algoritmo cuántico, se tiene que implementar usando puertas cuánticas.

Las puertas cuánticas, se **representan** mediante **matrices unitarias** U . Esto significa que conservan la probabilidad total de todos los posibles resultados (gracias a las propiedades de matrices unitarias), en otras palabras, no se pierde la información durante la operación.

Al aplicar una puerta cuántica a un estado cuántico, realizas $|\psi'\rangle = U|\psi\rangle$. Extendiendo dicha operación, se obtiene $|\psi'\rangle = \alpha'_0|0\rangle + \alpha'_1|1\rangle$. Aunque se aplique alguna puerta cuántica, se sigue manteniendo la ecuación $|\alpha'_0|^2 + |\alpha'_1|^2 = 1$.

Las puertas cuánticas para un único qubit son:

- **Pauli I:** Se le considera una extensión del conjunto de matrices de Pauli. La puerta viene denotada por la matriz identidad I .

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- **Pauli X:** Se conoce también como la puerta NOT. La representación matricial de dicha puerta es:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Cuando se aplica la puerta de Pauli X, el resultado es que el $|0\rangle$ y $|1\rangle$ se invierten.

- **Pauli Y:** La representación matricial de esta puerta es:

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

Si un qubit en superposición $|\psi\rangle = \alpha_1 |0\rangle + \alpha_2 |1\rangle$ entra en la puerta Y, el resultado es:

$$Y |\psi\rangle = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} = \begin{bmatrix} -i\alpha_2 \\ i\alpha_1 \end{bmatrix} = -i\alpha_2 |0\rangle + i\alpha_1 |1\rangle$$

- **Pauli Z:** La representación matricial es:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Cuando la puerta Pauli Z opera sobre un qubit, la fase cambia. Matemáticamente al entrar un qubit en estado de superposición $|\psi\rangle = \alpha_1 |0\rangle + \alpha_2 |1\rangle$ por la puerta Z, el resultado es el siguiente:

$$Z |\psi\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} = \begin{bmatrix} \alpha_1 \\ -\alpha_2 \end{bmatrix} = \alpha_1 |0\rangle - \alpha_2 |1\rangle$$

- **Puerta Hadamard (H):** Actúa sobre un qubit definido, al actuar, produce un estado de superposición como salida. La forma matricial es:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Si un qubit en estado de superposición $|\psi\rangle = \alpha_1 |0\rangle + \alpha_2 |1\rangle$ atraviesa esta puerta el resultado es:

$$H |\psi\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} \alpha_1 + \alpha_2 \\ \alpha_1 - \alpha_2 \end{bmatrix} = \left(\frac{\alpha_1 + \alpha_2}{\sqrt{2}} \right) |0\rangle + \left(\frac{\alpha_1 - \alpha_2}{\sqrt{2}} \right) |1\rangle$$

Capítulo 5

Procesamiento Lenguaje Natural (PLN)

5.1. ¿Qué es el procesamiento de lenguaje natural?

El PLN, es la rama de la IA encargada de la interacción entre computadoras y el lenguaje humano. Este campo integra la lingüística computacional, basada en el modelado de reglas del lenguaje, con modelos estadísticos y algoritmos de aprendizaje profundo (DL) y aprendizaje automático (ML).

El PLN forma parte a día de hoy de muchos motores de búsqueda, chatbots para atención al cliente (ya sea por escrito o por voz), y asistentes digitales como Alexa de Amazon o Siri de Apple.

El campo de la computación lingüística incluye tradicionalmente diversas fases para el análisis. Para el análisis de las palabras existen dos formas principales de hacerlo:

- **Análisis de dependencias:** Examina las relaciones gramaticales binarias entre las palabras, identificando, por ejemplo, qué palabra modifica a cuál o cuál es el sujeto y el verbo principal.
- **Análisis de constituyentes:** Construye un árbol sintáctico que representa la estructura gramatical anidada de una oración, descomponiéndola en sintagmas o frases ordenadas jerárquicamente.

5.2. Fases del PLN

Las fases para realizar el PLN se pueden observar en la siguiente imagen: A continuación, se detalla en qué consiste cada una de ellas:

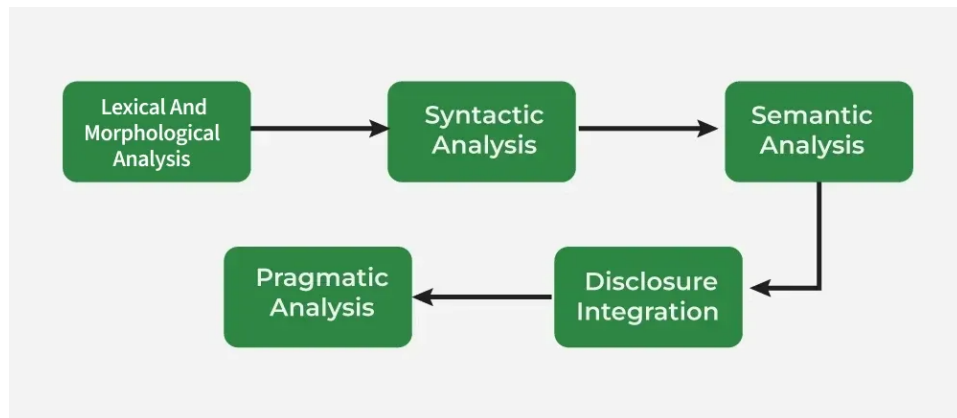


Figura 5.1: Imagen obtenida de <https://www.geeksforgeeks.org/machine-learning/phases-of-natural-language-processing-nlp/>

5.2.1. Preprocesamiento y Tokenización

Esta fase constituye el primer nivel de procesamiento y su objetivo es transformar el texto crudo en una secuencia de unidades discretas manejables, integrando tareas de análisis léxico y morfológico.

1. **Preprocesamiento:** Incluye la limpieza del texto, normalización (convertir a minúsculas), eliminación de caracteres especiales, y en ocasiones, la eliminación de *stopwords* (palabras muy frecuentes sin gran carga semántica como ".el", "de", "z"). También incluye técnicas de análisis morfológico como:

- *Lematización:* Convertir palabras a su forma base (e.g., "soy" → "ser").
- *Stemming:* Reducir palabras a su raíz (e.g., "niñero" → "niño").

2. **Tokenización:** Es el proceso crítico de segmentar el flujo de texto en unidades atómicas significativas o *tokens*.

Ejemplo: La oración *Me gusta programar* se transformaría en la lista de tokens: ["Me", "gusta", "programar"].

Dependiendo del modelo, los tokens pueden ser palabras completas, subpalabras (como en los modelos GPT) o incluso caracteres. Una vez tokenizado el texto, cada token se asocia a un índice numérico único en un vocabulario, preparando el terreno para la generación de *Embeddings*.

3. **Etiquetado gramatical (POS Tagging):** Asigna a cada token una categoría morfológica (sustantivo, verbo, adjetivo, etc.) basándose en

su definición y contexto. Esto permite desambiguar el rol sintáctico de cada palabra.

Ejemplo: Para los tokens del ejemplo anterior, el etiquetado resultante sería: ["Me" → Pronombre, "gusta" → Verbo, "programar" → Verbo].

5.2.2. Análisis sintáctico

El análisis sintáctico examina la organización de las palabras dentro de una oración para verificar que se cumple con las reglas gramaticales del idioma. El objetivo fundamental de esta fase es la generación de un **árbol de análisis sintáctico**, el cual representa jerárquicamente la estructura gramatical del texto analizado. Esta representación estructurada permite a los sistemas computacionales interpretar las relaciones de dependencia y jerarquía entre los distintos elementos léxicos. Las tareas principales que abarca son:

- **Etiquetado gramatical:** Corresponde al mismo proceso descrito anteriormente, validando la estructura en este nivel.
- **Resolución de ambigüedad sintáctica:** Se ocupa de determinar el significado correcto de una palabra basándose en su contexto gramatical. Por ejemplo, distinguir la acepción de la palabra *banco* en *Fui a sacar dinero al banco* (entidad financiera) frente a la frase *Me senté en el banco del parque* (sitio para sentarse y descansar).

5.2.3. Análisis semántico

Esta fase se centra en la coherencia lógica y la relevancia contextual del enunciado. Su objetivo principal es interpretar el significado de las palabras, analizando tanto su definición de diccionario como su variación según el contexto de uso. De este modo, se busca comprender el rol semántico de cada palabra para construir una representación lógica del mensaje. Las tareas fundamentales que abarca esta fase son:

1. **NER (NER):** Se ocupa de la identificación y clasificación de elementos clave en el texto, tales como nombres de personas, ubicaciones geográficas, organizaciones, etc.
Ejemplo: En la oración *Mercedes anunció un nuevo coche en Berlín*, el sistema NER identificaría a *Mercedes* como una *Organización* y a *Berlín* como un *Lugar*.
2. **WSD (WSD):** Identifica el significado correcto de las palabras en función del contexto.
Ejemplo: El término *Banco* puede interpretarse como una *entidad financiera* o como *un lugar donde sentarse* dependiendo del resto de palabras de la oración.

5.2.4. Integración del discurso

Este proceso analiza cómo las oraciones individuales se conectan y relacionan entre sí para entender el contexto global. El objetivo de esta fase es garantizar que el significado del texto mantenga su coherencia lógica a través de los distintos párrafos y frases. Los aspectos fundamentales de esta fase son:

- **Resolución de anáforas:** Consiste en identificar la conexión entre un pronombre y lo mencionado previamente en el texto.
Ejemplo: En la secuencia *Esther fue a comprar. Ella compró fruta*, el sistema debe vincular el pronombre *Ella* con *Esther*.
- **Referencias contextuales:** Aborda la interpretación de palabras o expresiones cuyo significado completo depende intrínsecamente de la información proporcionada en oraciones anteriores o posteriores.
Ejemplo: La frase *Fue un buen día* adquiere un significado concreto únicamente si se analiza dentro del contexto de la conversación.

5.2.5. Análisis pragmático

Esta fase final analiza más allá de la interpretación literal de las oraciones para inferir el significado real que se pretende transmitir. A diferencia del análisis semántico, que se ocupa del significado proposicional de las palabras, el análisis pragmático examina factores como el tono y el contexto. Las tareas principales de esta fase son:

- **Detección de intenciones:** El sistema debe ver el propósito de la comunicación (si es una petición, una queja, una orden, etc.).
Ejemplo: Ante la pregunta *¿Tienes hora?*, el análisis pragmático entiende que la intención real del usuario es saber qué hora es, no una pregunta de sí o no.
- **Interpretación de lenguaje figurado:** Se encarga de identificar metáforas, ironías o frases hechas.
Ejemplo: En la expresión *Echar una mano*, el análisis pragmático entiende que significa *ayudar a alguien*.

Capítulo 6

Procesamiento del Lenguaje Natural Cuántico (PLNC)

El procesamiento de datos clásicos utilizando algoritmos de aprendizaje automático a través de sistemas cuánticos, ha generado una línea de investigación de especial foto de interés estos últimos años. El QML se ha utilizado con distintos propósitos: profundizar en el uso de los fenómenos cuánticos para sistemas de aprendizaje, explorar la capacidad de los computadores cuánticos en aprender sobre datos cuánticos, y la posibilidad de reformular e implementar algoritmos de ML en hardware cuántico.

Como ocurrió anteriormente en el caso del ML clásico, el rápido crecimiento de los algoritmos de QML, tanto desde el lado del hardware como del software (en términos de dispositivos y algoritmos), ha involucrado al campo de PLN. Esto ha dado lugar al llamado PLNC, definido como la implementación del procesamiento del lenguaje natural en hardware cuántico.

6.1. Motivación para el uso de PLNC

La investigación del Procesamiento de Lenguaje Natural Cuántico (PLNC) está motivada fundamentalmente por las limitaciones físicas y computacionales de los modelos clásicos ante la creciente complejidad del lenguaje, especialmente tras la aparición de los LLMs. Los sistemas tradicionales requieren recursos masivos de tiempo y memoria para procesar grandes conjuntos de datos (corpus, como por ejemplo la Wikipedia). Además, los modelos PLN clásicos sufren degradación en precisión y fiabilidad al enfrentarse por ejemplo a la polisemia, es decir, cuando una palabra tiene distintos significados según el contexto, debido a las restricciones de las arquitecturas clásicas (frecuentemente basadas en estructuras de árbol) [10].

Para afrontar estos retos, el PLNC aprovecha fenómenos de la mecánica cuántica como la **superposición** y el **entrelazamiento**. Al operar en el espacio de *Hilbert*, esta arquitectura permite almacenar múltiples significados

y contextos simultáneamente en un solo registro y ejecutar operaciones en paralelo. Esto no solo ofrece una ventaja exponencial acelerando la computación [31], sino que también proporciona una 'expresividad' superior, capturando las dependencias semánticas del lenguaje humano de una manera que los espacios numéricos clásicos replican con menor eficiencia.

No obstante, hay que tener en cuenta que, aunque teóricamente el PLNC puede resolver estos problemas, aún no se ha podido realizar una comparación a gran escala en hardware real (sin usar simuladores) frente a la computación clásica. Los modelos cuánticos actuales se han implementado sobre corpus pequeños, por lo que todavía no representan la escala completa que maneja un LLM moderno.

6.2. Codificación de información clásica en estados cuánticos

Una de las hipótesis fundamentales que motiva la intersección entre el procesamiento de lenguaje natural y la computación cuántica es la existencia de una relación estructural directa entre las características lingüísticas (sintáctica y semántica) y los estados cuánticos.

El framework Distributional Compositional Categorical (DisCoCat) [34] es el más conocido en formalizar esta relación. Nace de proponer la unificación de los dos enfoques clásicos del significado:

- **Distribucional:** Basado en la estadística y el contexto, que es la base de los vectores de palabras modernos.
- **Composicional:** Basado en la estructura gramatical, donde el significado de una frase depende de sus partes y de cómo se combinan (reglas sintácticas).

La relevancia de DisCoCat para el PLNC radica en que modela el lenguaje utilizando diagramas de palabras y productos tensoriales, una estructura matemática que es a la que describe a su vez la computación cuántica. Esto sugiere que los ordenadores cuánticos son, de manera nativa, idóneos para procesar el lenguaje natural.

En la imagen, las cajas representan el significado de las palabras que se transmiten mediante los cables", esto es similar a la técnica de Dependency Parse Tree (DPT) muy popular en la literatura lingüística. En la frase, el sujeto es "Dani", mientras que "pizza.^{es} el objeto, ambas palabras se relacionan mediante el verbo *comez* la combinación de estas palabras, crean el significado conjunto de la frase. Con DisCoCat, el significado de la oración se construye utilizando una gramática preagrupada mediante composición de productos tensoriales. Es decir, quedaría representado de la siguiente manera:

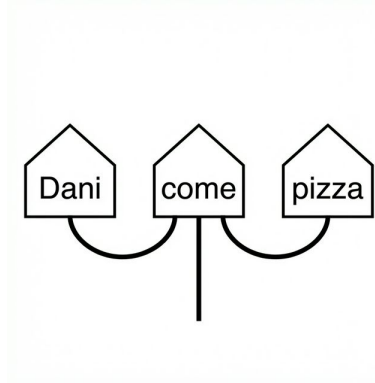


Figura 6.1: Diagrama de la oración "Dani come pizza."^{en} el formalismo DisCoCat.

$$(\overrightarrow{\text{Dani}} \otimes \overrightarrow{\text{subj}}) \otimes \overrightarrow{\text{come}} \otimes (\overrightarrow{\text{pizza}} \otimes \overrightarrow{\text{obj}})$$

Este vector generado en el espacio de productos tensoriales, se puede considerar como el significado de la oración "Dani come pizza". Por ende, se dice que este framework lo ha expresado en términos de computación cuántica.

Cabe destacar que, aunque el modelo DisCoCat original funciona perfectamente sin referencias explícitas a la teoría cuántica (a pesar de originarse en el formalismo de la Categorical Quantum Mechanics (CQM)). La novedad yace en el argumento de que el PLNC puede considerarse "nativo cuántico".

Esto se debe a que la teoría cuántica y el lenguaje natural comparten una misma estructura de interacción y el uso fundamental de espacios vectoriales para describir sus estados. Esta estructura de interacción determina por completo la estructura los procesos, incluyendo la especificación de los espacios donde "viven" los estados. Esta coincidencia estructural implica que el lenguaje natural podría encajar de manera más natural y eficiente en hardware cuántico que en hardware clásico.

6.2.1. Técnicas de codificación de datos clásicos a estados cuánticos

Al igual que en computación clásica la información necesita ser codificada de manera que los modelos pueda entender los datos que se le proporciona y así trabajar con ellos, la computación cuántica necesita poder codificar los datos de la computación clásica (texto en este caso, ya que estamos dentro del ámbito de PLN y PLNC), a estados cuánticos. Para ello existe diversas técnicas que son:

- **Basis Encoding:** Utiliza n qubits. Complejidad $O(1)$. Simple, entradas cortas y simbólicas.
- **Angle Encoding:** Utiliza n qubits. Complejidad $O(n)$. Resiliente al ruido, vectores de características pequeños.
- **Amplitude Encoding:** Utiliza $\log_2(n)$ qubits. Complejidad $O(2^n)$. Eficiente en memoria, vectores de frases completas.
- **Hamiltonian Encoding:** Utiliza $\log_2(n)$ qubits. Complejidad $O(t)$. Estructura de grafos, análisis sintáctico (parsing), traducción.

De todas estas técnicas, se ha optado por implementar la técnica de *Basis Encoding* [24]. Este tipo de codificación se utiliza principalmente cuando un número real debe ser matemáticamente manipulado en un algoritmo cuántico. Su funcionamiento consiste en representar un número real como un número binario y luego transformarlo en un estado cuántico en la base computacional.

El estado cuántico embebido es la traducción bit a bit de una cadena binaria a los estados correspondientes de los subsistemas cuánticos. Por lo tanto, un bit de información clásica está representado por un subsistema cuántico. Actuar sobre características binarias codificadas como qubits otorga mayor flexibilidad computacional para diseñar algoritmos cuánticos.

Supongamos que queremos codificar la frase "María estudia medicina" utilizando *Basis Encoding*. Primero, definimos un vocabulario simplificado donde asignamos un índice entero único a cada palabra y fijamos una longitud de $\tau = 5$ bits por palabra (lo que permitiría un vocabulario de hasta $2^5 = 32$ palabras).

- $María \rightarrow \text{Índice } 3 \rightarrow 00011$
- $estudia \rightarrow \text{Índice } 12 \rightarrow 01100$
- $medicina \rightarrow \text{Índice } 21 \rightarrow 10101$

El vector de características de la frase corresponde entonces a la concatenación de estas secuencias binarias: $b = 00011 \ 01100 \ 10101$.

Finalmente, esta secuencia se representa mediante el estado cuántico conjunto en la base computacional:

$$|\psi\rangle_{\text{frase}} = |00011\rangle \otimes |01100\rangle \otimes |10101\rangle = |00011 \ 01100 \ 10101\rangle$$

De esta forma, hemos codificado una frase de lengua clásica directamente en un estado cuántico en la base computacional.

6.3. Circuitos Variacionales Cuánticos (CVC) como modelos de aprendizaje

En 2016, se tuvo acceso al primer ordenador cuántico en la nube aunque el ruido y las limitaciones de qubits de las que se disponía, hacía que fuera difícil implementar algoritmos cuánticos.

A pesar de ello, hubo una emoción muy grande con lo que se podría hacer con estos nuevos ordenadores llamados NISQ. Estos nuevos ordenadores tenían entre 50 y 100 Qubits, lo que les permitía sobrepasar en rendimiento a los mejores superordenadores clásicos en algunas áreas matemáticas [2, 35].

Los CVC nacieron como la mejor estrategia para hacer frente a los ordenadores NISQ. Debido a las limitaciones de estos dispositivos, se utilizó la estrategia de optimización basado en el aprendizaje. No son más que una analogía cuántica a los técnicas existentes del campo de ML como pueden ser las redes neuronales.

En definitiva, son algoritmos cuánticos que tienen parámetros libres [19]. Al igual que cualquier otro circuito cuántico estándar, se necesita hacer uso de estos tres conceptos básicos:

1. Preparación con un **estado inicial** fijo.
2. Un **circuito cuántico** $U(\theta)$ parametrizado con un conjunto de parámetros libres θ .
3. **Medición** de un observable $\hat{B}$ en la salida. Este observable puede estar compuesto por varios observables cada uno de manera local a un cable del circuito, o bien, a un conjunto de cables del circuito.

Normalmente, los valores esperados $f(\theta) = \langle 0|U^\dagger(\theta)\hat{B}U(\theta)|0\rangle$ de uno o varios circuitos, definen un coste escalar para la tarea específica. Los parámetros libres $\theta = (\theta_1, \theta_2, \dots)$ son ajustados para optimizar esta función de coste.

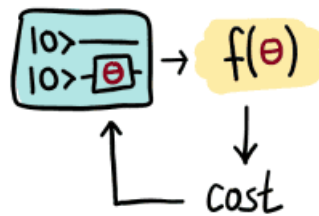


Figura 6.2: Coste de un Circuito Cuántico Variacional, imagen obtenida de: [19]

Los circuitos cuánticos variacionales son entrenados mediante un algoritmo de optimización clásico que realiza consultas al hardware cuántico. La optimización suele ser un esquema iterativo que busca mejores candidatos para los parámetros en cada paso.

Los parámetros variacionales θ , posiblemente junto con un conjunto adicional de parámetros no adaptables $x = (x_1, x_2, \dots)$, entran en el circuito cuántico como argumentos para las puertas del circuito. Esto nos permite convertir *información clásica* (los valores θ y x) en *información cuántica* (el estado cuántico $U(x; \theta)|0\rangle$). Como veremos en el ejemplo a continuación, los parámetros de puerta no adaptables suelen desempeñar el papel de *entradas de datos* en el aprendizaje automático cuántico.

La información cuántica se convierte de nuevo en información clásica evaluando el valor esperado del observable $\hat{B}$,

$$f(x; \theta) = \langle \hat{B} \rangle = \langle 0 | U^\dagger(x; \theta) \hat{B} U(x; \theta) | 0 \rangle.$$

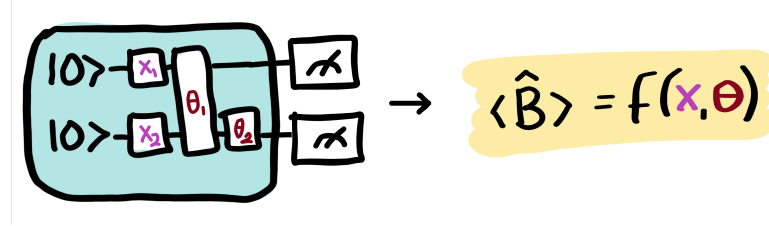


Figura 6.3: Circuito variacional cuántico con parámetros modificables x y no modificables θ . Imagen obtenida de: [19]

Capítulo 7

Representación Vectorial de Palabras (Word Embeddings)

Los word embeddings son una de las técnicas más usadas en NLP. Gracias a los word embeddings, existen lenguajes de modelo como GPT-3, GPT-4, etc.

Estos algoritmos son rápidos y pueden generar secuencias de lenguaje y realizar otras tareas derivadas con alta precisión, incluyendo la comprensión contextual, propiedades semánticas y sintácticas, así como la relación lineal entre las palabras.

En esencia, estos modelos utilizan los embeddings como un método para extraer patrones de secuencias de texto o voz. Pero, ¿cómo lo hacen?

En las siguientes secciones se explorará en mayor profundidad los word embeddings así como conceptos más avanzados de estos.

7.1. Introducción al concepto de *Embedding*

De manera intuitiva, un *embedding* de texto es una representación matemática que proyecta un fragmento de texto en un espacio latente de alta dimensionalidad. En este espacio, la posición del texto se define mediante un vector, donde cada componente del vector corresponde a una dimensión. Al igual que en un plano cartesiano se tienen dos dimensiones, en un embedding, se suele trabajar con muchas más dimensiones para representar una palabra en el espacio[11]. Por ejemplo, el modelo de **OpenAI** *text-embedding-ada-002* el cual tiene un espacio de dimensión de 1536. Es decir, cada vector está formado por 1536 componentes.

Un ejemplo de cómo se verían los valores que toma cada dimensión es el siguiente:

“A text embedding is a piece of text projected into a high-dimensional latent space. The position of our text in this space is a vector, a long sequence of numbers. Think of the two-dimensional cartesian coordinates from algebra class, but with more dimensions—often 768 or 1536.” [11]

Al procesar el párrafo anterior mediante el modelo *text-embedding-ada-002* mencionado, se obtiene la gráfica de la Figura 7.1, donde cada barra vertical representa el valor asignado a una de sus 1536 dimensiones.

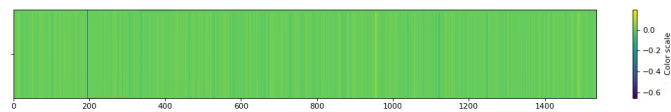


Figura 7.1: Representación gráfica de los valores en las 1536 dimensiones de un embedding generado por el modelo *text-embedding-ada-002*. Fuente: [11].

Un **espacio de embedding** o **espacio latente** se define matemáticamente como un espacio donde los elementos están posicionados de manera más cercana uno del otro cuando tienen valores parecidos, mientras que si difieren dichos elementos, se posicionan más lejos entre ellos. En el caso del PLN, frases que son semánticamente más similares a otras, tendrán un vector de embedding similar por lo que estarán más juntas en el espacio.

Aunque la comparación entre frases es una de las aplicaciones más conocidas, los *embeddings* también se aplica en modelos de ML y en arquitecturas de redes neuronales complejas.

7.1.1. Embeddings de palabras (*Word Embeddings*)

Mientras que el concepto general de *embedding* puede aplicarse a frases completas (como en el ejemplo introducido), su uso más extendido e histórico se encuentra en su aplicación a nivel atómico: las palabras. Un *embedding* de palabras, es un vector de longitud fija el cual su único cometido, es representar una única palabra o *token* [1].

El diseño y creación de estos vectores se clasifica principalmente en dos grandes familias estratégicas, dependientes de sus algoritmos de generación: los **modelos basados en predicciones** y los **modelos basados en conteos**.

Modelos basados en predicciones

Esta técnica de representación surgió fuertemente ligada al desarrollo de los MNLs. A diferencia de los enfoques estadísticos tradicionales de recuento,

un modelo predictivo se centra en aprender el significado de un término de forma prospectiva, es decir, “adivinando”.

Para lograrlo, emplean una red neuronal sencilla dotada de una ventana de contexto de tamaño fijo y móvil (por ejemplo, avanzando de cinco en cinco palabras). A medida que lee el texto, el modelo intenta predecir el significado de una palabra central en base a las palabras vecinas de esta ventana, o a la inversa, predecir el contexto apoyándose únicamente en dicho término central.

El aspecto fundamental en todo esto, es que tras finalizar el entrenamiento, el objetivo de ML deja de ser la capacidad del modelo para adivinar con exactitud. En su lugar, de la red neuronal, se extraen los **pesos y valores** que ha ajustado internamente. Estos valores que fueron ajustados durante el entrenamiento, se convierten directamente en el *word embedding*. En consecuencia, el mecanismo iterativo garantiza que aquellas palabras empleadas en contextos idénticos acaben reteniendo representaciones vectoriales prácticamente iguales.

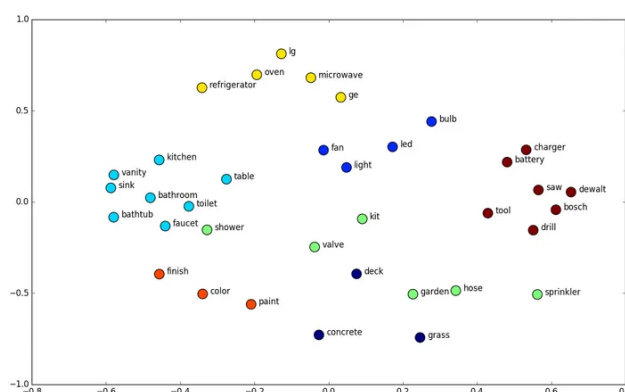


Figura 7.2: Representación visual de un modelo basado en predicciones para generar embeddings de palabras. Fuente: <https://neptune.ai/blog/word-embeddings-guide>.

Una de las arquitecturas más representativas dentro de la predicción de *embeddings* es la famosa familia algorítmica **Word2Vec**, la cual se detalla de forma exhaustiva en el próximo capítulo.

Modelos basados en conteos

Por su parte, la segunda categoría de modelos hace uso de la estadística global a lo largo de un documento completo (*corpus*). Estos modelos se aprovechan de los recuentos de coocurrencia (palabra y contexto) estructurando toda la información como grandes *matrices de palabra-contexto*.

El funcionamiento radica en la creación de una matriz gigantesca donde se contabiliza el número de apariciones conjuntas en las que una palabra acompaña a todo un posible contexto del propio texto. No obstante, si el conjunto de datos cuenta con n palabras distintas, las dimensiones de la matriz ascenderán a un total de $n \times n$. Esto acarrea severamente un cuello de botella de almacenamiento. Adicionalmente, la inmensa mayoría de cruces de dicha matriz representarán palabras que jamás han coexistido en el texto (por ejemplo, .°vejaz "pantalla"), llenando de ceros el modelo de datos. Esta estructura tan ineficiente es conocida como **matriz dispersa**.

Para solventar esta carencia y la falta de memoria, se reduce la dimensionalidad a través de complejas operaciones de álgebra lineal, como puede ser la Descomposición en Valores Singulares (DVS). Con esto logran sintetizar enormes matrices de ceros, transformándolos en **vectores cortos y densos**. De tal modo, que los conceptos globalmente equivalentes retengan valores numéricamente similares sin desperdiciar memoria.

La **mayor ventaja** de esta táctica estadística contable, frente al sistema predictivo, reside precisamente en que es capaz de aprovechar la información global de todo el *corpus*, un aspecto estadístico que se le escapa a las ventanas de lectura en las redes neuronales por definición.

El modelo híbrido con más popularidad en este ámbito es el de **GloVe (Global Vectors)**. Desarrollado por un equipo de investigadores de la Universidad de Stanford, logra exitosamente agrupar los beneficios numéricos de las matrices grandes de conteo global, con las altas capacidades de optimización paramétrica propias de los modelos del ML.

7.2. Mecanismos geométricos de los embeddings

Una vez comprendido que un embedding proyecta palabras en un espacio continuo donde la cercanía espacial equivale a similitud semántica, surgen dos interrogantes fundamentales en el diseño de estos sistemas: ¿cómo se estructuran los ejes de este espacio para encapsular el significado complejo del lenguaje? y ¿cómo se calcula con precisión matemática la distancia entre dos vectores?

Para responder a la primera cuestión, es necesario analizar el papel de la **información latente** [11], la cual permite pasar de representaciones dispersas a vectores densos basados en atributos ocultos. Para la segunda, se debe explorar **métricas de distancia** específicas que sean efectivas en espacios de tantas dimensiones.

7.2.1. Métricas de Distancia

Como se explicó con anterioridad, una de las características principales de un embedding representado en el espacio, es que mantiene la distancia. Los vectores de alta dimensionalidad que se utilizan en los embeddings de

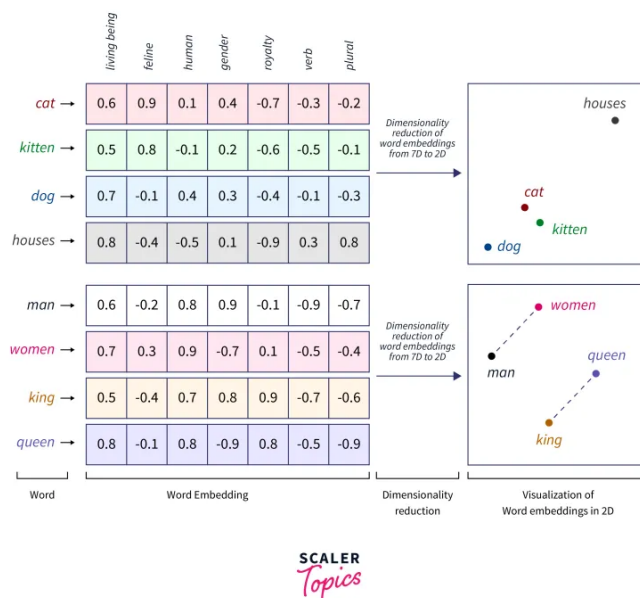


Figura 7.3: Representación visual de un modelo basado en conteos para generar embeddings de palabras. Fuente: <https://www.scaler.com/topics/tensorflow/tensorflow-word-embeddings/>.

palabra (así como en los LLMs), no son intuitivos. Pero la intuición espacial se mantiene igual mientras reducimos la escala de estos.

Un ejemplo básico es el siguiente: imagina un plano de planta bidimensional de una biblioteca de una sola planta. Los usuarios de la biblioteca son amantes de los gatos, de los perros, o se sitúan en algún punto intermedio. El objetivo es colocar los libros sobre gatos cerca de otros libros de gatos, y los libros sobre perros cerca de otros libros de perros.

El enfoque más sencillo se conoce como el modelo de bolsa de palabras. Consiste en establecer un eje de perros a lo largo de una pared y un eje de gatos perpendicular a él. A continuación, se cuentan las ocurrencias de las palabras "gato" "perro" en cada libro y se ubica dicho libro en su punto correspondiente dentro del sistema de coordenadas ($\text{perro}_x, \text{gato}_y$).

Pensemos ahora en un sistema de recomendación sencillo. A partir de la selección anterior de un libro, ¿qué se podría sugerir a continuación? Bajo la suposición (muy simplificada) de que estas dos dimensiones capturan adecuadamente las preferencias del lector, basta con buscar el libro que se encuentre más cerca. En este caso, el sentido intuitivo de cercanía es la distancia euclídea —el camino más corto entre dos libros—:

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

Siguiendo el razonamiento lógico, si se analiza a través de la distancia euclídea, se pondrá el libro (perro₅, gato₁) más cerca del libro (perro₁, gato₅) que de un libro mucho más extenso o enciclopédico como (perro₁₀₀, gato₁). Esto plantea un grave problema semántico: un lector de libros cortos de perros con proporción (5, 1) seguramente prefiera a continuación una enciclopedia gigante sobre perros de (100, 1) antes que un libro corto sobre gatos de (1, 5). La métrica euclídea penaliza severamente la longitud en palabras del libro (magnitud) omitiendo por completo el reparto del tema de este (la proporción).

Cuando resulta más relevante concentrarse en los pesos relativos temáticos que en sus magnitudes absolutas, los vectores se normalizan. Esto se logra dividiendo en ambos la cantidad de apariciones de perros y gatos para obtener la conocida **similitud coseno** [22]. De manera más visual, este cálculo equivale geoméricamente a proyectar todos los puntos disponibles en el espacio sobre una circunferencia de radio unidad (es decir, que el módulo de todos los vectores es 1) y medir la distancia a lo largo del arco que los une. Al ignorar la longitud del documento original, se evalúa únicamente la dirección del vector respecto al origen, logrando una métrica de similitud mucho más robusta y natural al lenguaje humano.

Matemáticamente, esta métrica se define mediante el cociente del producto escalar de dos determinados vectores entre el producto de sus magnitudes:

$$\text{similitud}(\mathbf{p}, \mathbf{q}) = \cos(\theta) = \frac{\mathbf{p} \cdot \mathbf{q}}{\|\mathbf{p}\| \|\mathbf{q}\|} = \frac{\sum_{i=1}^n p_i q_i}{\sqrt{\sum_{i=1}^n p_i^2} \sqrt{\sum_{i=1}^n q_i^2}}$$

Un pensamiento intuitivo para entender la fórmula de similitud coseno, es que si dos vectores apuntan en la misma dirección, el ángulo entre ellos es 0 por lo que el coseno es 1. Si son ortogonales, significa que no comparten una 'direccionalidad', el coseno es 0.

En el contexto que nos encontramos de embeddings de palabras, hay que pensar que cada vector actúa como una flecha que apunta desde el origen hasta la posición de la palabra en nuestra librería imaginada. Palabras que son similar semánticamente, tendrán vectores que apuntan en direcciones similares, es decir, la similitud de coseno será mayor.

7.2.2. Información Latente

Cualquier texto suele emplear sinónimos o términos relacionados para referirse a un mismo concepto; por ejemplo, volviendo al mismo ejemplo de an-

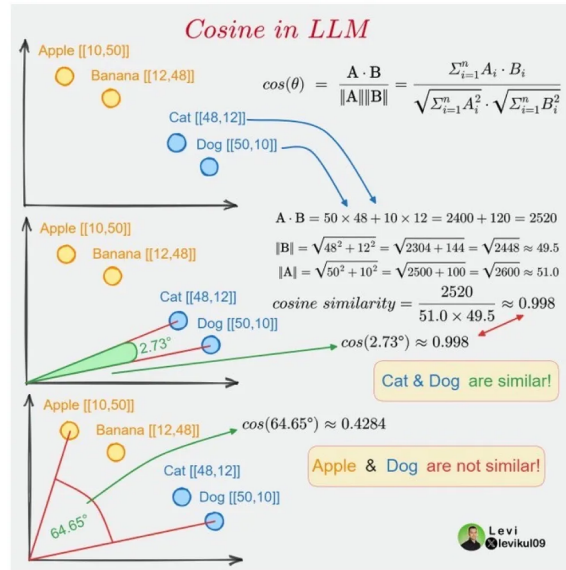


Figura 7.4: Ejemplo visual de la representación de vectores y su cercanía empleando la similitud coseno. Fuente: <https://x.com/levikul09>.

tes, un libro sobre perros utilizará frecuentemente palabras como canino”. En un modelo básico de bolsa de palabras, incorporar este nuevo vocablo requiere añadir una dimensión perpendicular adicional a las ya existentes, transformando nuestro anterior sistema de coordenadas en $(\text{perro}_x, \text{gato}_y, \text{canino}_z)$.

Bajo este mismo principio, resulta muy sencillo continuar agregando nuevas dimensiones para representar más palabras, consiguiendo un espacio multidimensional $(\text{perro}_x, \text{gato}_y, \text{canino}_z, \text{felino}_w)$. Sin embargo, este enfoque tan básico conlleva dos grandes problemas que imposibilitan su aplicación a gran escala: rompe de inmediato la interpretabilidad espacial (como lo era la sencilla idea inicial de una biblioteca bidimensional) y provoca una explosión en los requisitos necesarios para el cómputo.

Resulta evidente que, a nivel computacional, esta aproximación es extremadamente ineficiente a medida que aumenta el tamaño del vocabulario. El español, por ejemplo, cuenta con más de 93.000 palabras según el Diccionario de la Real Academia Española [4]. Manejar vectores con decenas de miles de dimensiones implica calcular una ingente cantidad de combinatorias. Además, se agrava el problema de la dispersión de datos: en un documento dado, la gran mayoría del vocabulario total no aparecerá, rellenando los vectores con innumerables ceros que penalizan drásticamente el rendimiento de los modelos estadísticos y de aprendizaje automático.

La solución a la alta dimensionalidad radica en abstraer el conteo literal hacia ejes que representen verdaderos atributos semánticos. Si el modelo es capaz de identificar la relación semántica entre “gatoz” “felino”, podría

347.3. Paralelismo entre Espacio Semántico y Espacio de Hilbert

agrupar la contribución de ambas en una única variable latente compartida. Esto no solo comprime el tamaño de los vectores, haciendo a los algoritmos mucho más eficientes y veloces, sino que también genera una representación geométrica mucho más afín a la intuición humana.

De la misma forma, términos más amplios como "mascota.^o" "mamífero" representan características semánticas compartidas simultáneamente por "perroz" "gato". En un espacio latente bien optimizado, estas palabras genéricas actúan como atributos comunes que vinculan y aproximan ambos animales en ciertas dimensiones del espacio. Por otro lado, si la elevada reducción dimensional provocara una superposición excesiva donde términos que debieran ser distintos terminen colapsando —por ejemplo, confundándose ambos en el mismo punto exacto—, esta carencia de precisión podría mitigarse introduciendo nuevos ejes latentes. Estos nuevos atributos ocultos de diferenciación (por ejemplo, proyectando qué palabras se relacionan estrechamente con "mesa" frente a los animales vivos mencionados anteriormente) permiten desenredar los agrupamientos de datos semánticos complejos.

7.3. Paralelismo entre Espacio Semántico y Espacio de Hilbert

Como se vio en capítulos anteriores, los estados en computación cuántica están representados por vectores definidos en un espacio complejo, el cual se conoce formalmente como **espacio de Hilbert** ($\mathcal{H}$). Por su parte, en el PLN clásico se modela el significado de las palabras mapeándolas a vectores en un espacio vectorial puramente real ($\mathbb{R}^n$).

A partir de esta analogía natural, surge la posibilidad de modelar las palabras y sus significados como estados dentro del propio espacio de Hilbert [33].

7.3.1. Codificación multidimensional: Forma y Contenido

En un espacio semántico tradicional, el significado de una palabra adquiere "sustancia.^a" ubicarse en unas coordenadas determinadas que definen su forma. Sin embargo, al restringirse al uso exclusivo de los números reales, el PLN clásico queda comúnmente limitado a capturar únicamente su *semántica distribucional* (las estadísticas y frecuencias con las que coocurre junto a otras palabras).

Al trasladar el procesamiento de lenguaje al dominio cuántico de Hilbert, las propiedades exclusivas de los números complejos permiten unificar distintos niveles de información semántica en un único vector de estado. Esto logra vincular el uso externo del lenguaje (su *forma*) directamente con su fundamento conceptual intrínseco (su *contenido*). Para ello, se propone una división explícita de las dimensiones de almacenamiento:

- **Componente Real:** Se utiliza de la misma forma para codificar la *semántica distribucional*. Captura el uso estadístico clásico, determinando el comportamiento de la palabra estrictamente en base a su contexto y al número de interacciones o apariciones que realiza con respecto a las palabras que le rodean en los distintos documentos analizados.
- **Componente Imaginaria:** Se reserva de forma ortogonal para alojar la *semántica referencial u ontológica*. Representa los fundamentos preexistentes, los conceptos formales o el núcleo estricto que designa la propia palabra (el significado que podría extraerse de ontologías taxonómicas u organizaciones como base de conocimientos, por ejemplo *WordNet* o diagramas *SNOMED-CT*).

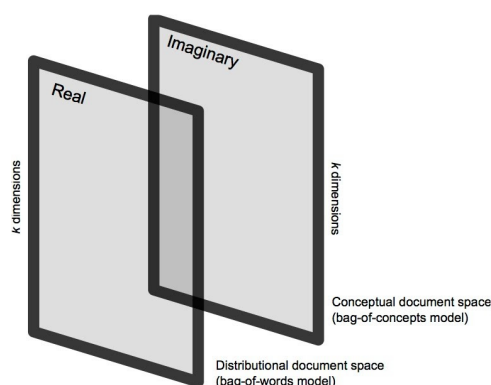


Figura 7.5: Esquema del espacio documental complejo vectorial unificando la semántica distribucional y referencial en el espacio de Hilbert. Fuente: [33].

7.3.2. Afinidad Cuántica, Superposiciones y Fases

En el apartado clásico se demostró que la afinidad semántica puede medirse mediante distintas distancias sobre un espacio real continuo. No obstante, al emplear el espacio de Hilbert esta medición no solo relaciona dos palabras distintas, sino que va un paso más allá. Al tratar con números complejos, es posible medir la distancia cuantitativa que existe entre la parte estadística de uso de la palabra (su componente real) y su definición conceptual referencial alojada en su componente imaginaria.

Para este análisis se evalúa la **fase o ángulo** que se forma interactivamente entre ambas componentes, logrando que el producto interno refleje tanto la componente real como imaginaria del contenido.

Capítulo 8

Implementación Clásica: Algoritmo Word2Vec

Word2Vec, cuyo nombre proviene de la contracción de *Word to Vector*, es un modelo de ML que permite representar palabras como vectores de grandes dimensiones. Su principal virtud reside en que las relaciones semánticas y sintácticas entre palabras quedan codificadas como proximidad geométrica entre dichos vectores, permitiendo así capturar el significado de forma numérica y operable.

Fue desarrollado y publicado por Google en 2013 [8], además en esa misma fecha, se introdujo este modelo en su motor de búsqueda. Desde entonces, el modelo ha sido ampliamente adoptado y ha dado lugar a múltiples mejoras y variantes de rendimiento.

En este capítulo se explorarán la arquitectura del modelo, su función de optimización y el proceso de entrenamiento que permite generar el espacio vectorial latente en el que las palabras quedan representadas.

8.1. Arquitectura del modelo

Word2Vec aprende las representaciones de las palabras procesando grandes volúmenes de texto y capturando el contexto en el que aparece cada término [14]. Su arquitectura se fundamenta en una red neuronal superficial (*shallow neural network*), que a diferencia de las redes profundas (*deep neural networks*), dispone únicamente de una capa oculta entre la capa de entrada y la de salida. Este diseño proporciona una ventaja clave: permite entrenar el modelo de forma eficiente sobre corpus de gran tamaño, manteniendo un coste computacional razonable.

El resultado del entrenamiento es la asignación de un vector de cientos de dimensiones a cada palabra del vocabulario. La posición de cada vector en ese espacio no es arbitraria, es decir, palabras empleadas en contextos similares acaban próximas entre sí, mientras que palabras semánticamente

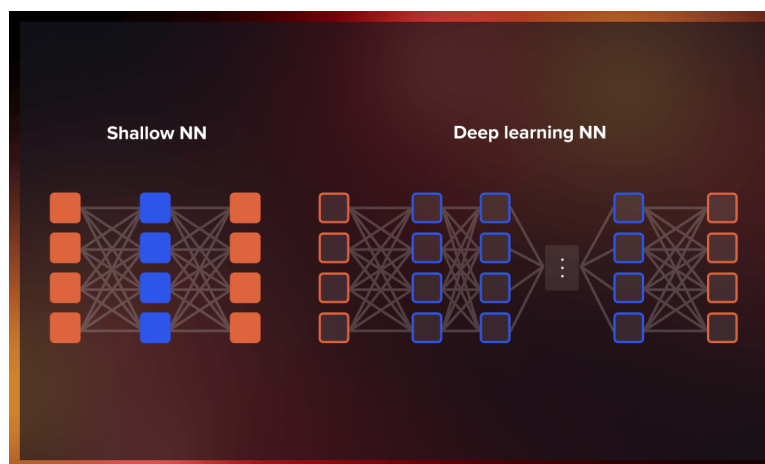


Figura 8.1: Comparativa entre una red neuronal superficial (Word2Vec) y una red neuronal profunda. Fuente: [14].

distantes se alejan geométicamente.

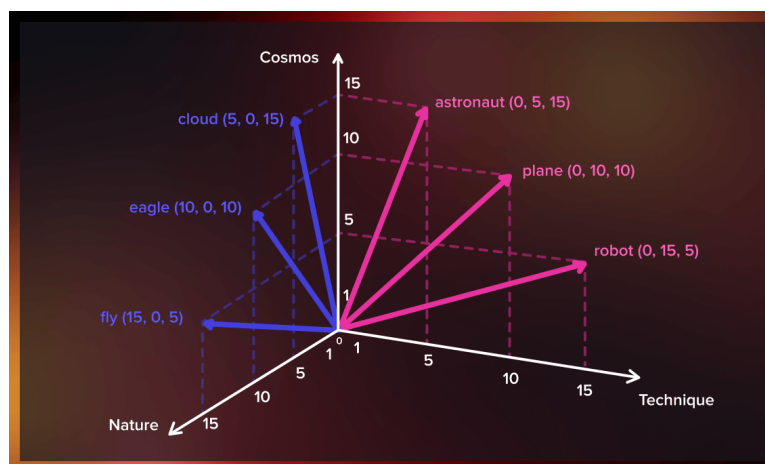


Figura 8.2: Representación vectorial de palabras en el espacio latente generado por Word2Vec. Fuente: [14].

Una consecuencia directa de esta estructura es que los vectores admiten operaciones algebraicas con significado semántico. El ejemplo más conocido de esta propiedad es la siguiente analogía entre género y realeza:

$$\vec{v}(\text{Rey}) - \vec{v}(\text{Hombre}) + \vec{v}(\text{Mujer}) \approx \vec{v}(\text{Reina})$$

En esta expresión, al restar al vector de *Rey* la componente que codifica la masculinidad y sumar la que codifica la feminidad, el vector resultante converge hacia la representación de *Reina*. Esta propiedad es inherente a

la forma natural del entrenamiento: como *Rey* y *Reina* comparten contextos de realeza pero difieren en los contextos de género, el modelo aprende implícitamente a separar ambas dimensiones dentro del espacio vectorial.

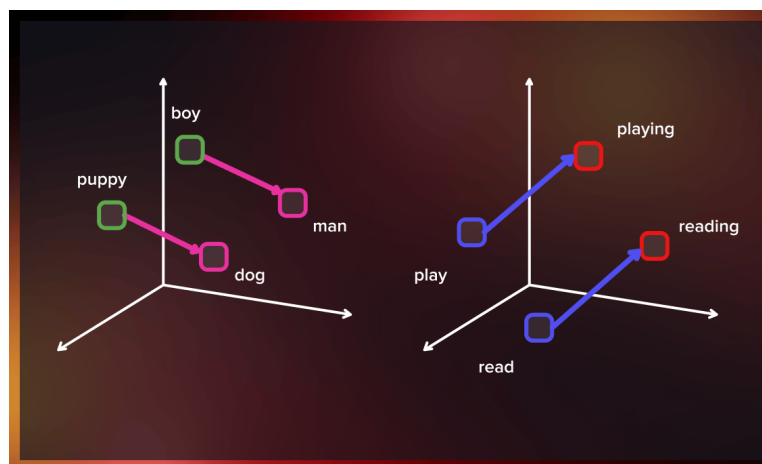


Figura 8.3: Representación geométrica de las similitudes semánticas entre palabras en el espacio vectorial de Word2Vec. Fuente: [14].

8.1.1. Modelos de entrenamiento: CBOW y Skip-gram

Word2Vec ofrece dos variantes arquitectónicas para aprender los vectores de palabras: el modelo **CBOW** y el modelo **Skip-gram**. Ambos comparten la misma red neuronal superficial como base, pero se diferencian en el aprendizaje, qué se usa como entrada y qué se predice como salida.

En este trabajo se ha optado por implementar el modelo **Skip-gram**, ya que, como se verá en los capítulos siguientes, sus características lo hacen especialmente adecuado para la adaptación al entorno cuántico propuesto.

Continuous Bag of Words (CBOW)

El modelo CBOW toma como entrada las palabras del contexto que rodean a una posición central y predice cuál es la palabra que debería ocupar dicha posición. En otras palabras, dadas las palabras vecinas, el modelo intenta adivinar la palabra central.

Este enfoque resulta eficiente cuando el corpus es grande y las palabras frecuentes tienen representaciones ricas. Sin embargo, tiende a promediar la información del contexto, lo que puede penalizar los valores semánticos de palabras menos frecuentes.

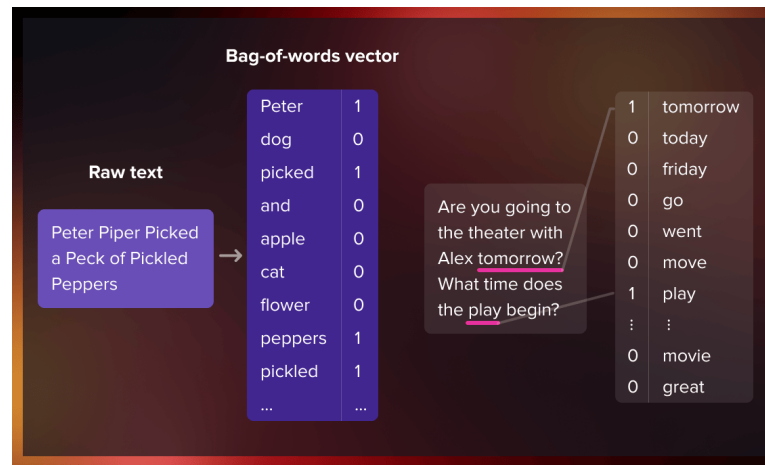


Figura 8.4: Ejemplo del funcionamiento del modelo CBOW: a partir de las palabras de contexto circundantes se predice la palabra objetivo central. Fuente: [14].

Skip-gram

Skip-gram es un modelo diseñado para hacer lo opuesto a CBOW. En lugar de predecir la palabra central a partir del contexto que tiene en su ventana, toma una única palabra central como entrada e intenta predecir las palabras que aparecen a su alrededor.

El proceso de entrenamiento de esta red neuronal superficial funciona de la siguiente manera: dada la palabra central (la palabra de entrada), se selecciona aleatoriamente una palabra de su contexto inmediato. La red calcula cuál es la probabilidad de que cada palabra del vocabulario sea una palabra cercana.^a la palabra de entrada [15].

Los resultados de estas probabilidades indican cuán probable es que cada término del vocabulario aparezca en las proximidades de la palabra de entrada. Por ejemplo, si la palabra de entrada proporcionada a la red ya entrenada es *reino*, las probabilidades serán considerablemente mayores para palabras como *España* o *Inglaterra* que para términos sin relación semántica como *lápiz* o *mango*.

Detalles del modelo básico Skip-gram

Dado que una red neuronal no puede procesar cadenas de texto directamente, cada palabra se representa mediante un *vector one-hot* (una forma de codificación sencilla aunque no la única). Consiste en un vector de tantas componentes como palabras tiene el vocabulario (por ejemplo, 10.000), donde únicamente la posición correspondiente a la palabra de entrada vale 1 y el resto valen 0.

La capa de salida de la red produce también un vector de 10.000 componentes, donde cada valor refleja la probabilidad de que la palabra correspondiente sea una palabra cercana a la entrada. No existe función de activación en la capa oculta, pero la capa de salida aplica *softmax* para normalizar las probabilidades.

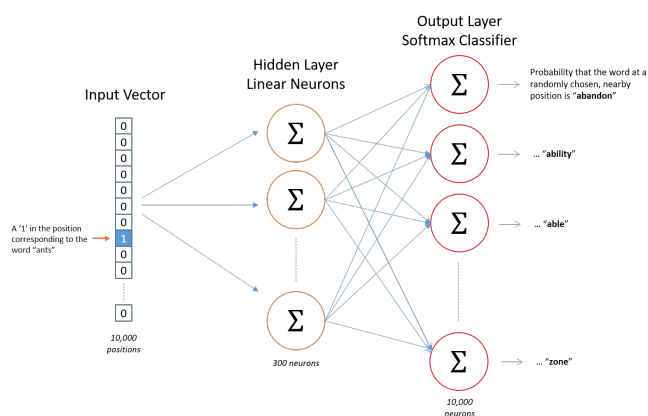


Figura 8.5: Arquitectura de la red neuronal del modelo Skip-gram. Fuente: [15].

Durante el entrenamiento, tanto la entrada como la salida de la red son vectores *one-hot*. Sin embargo, al evaluar la red con una palabra de entrada, el vector de salida ya no es *one-hot*, sino una distribución de probabilidad sobre todo el vocabulario.

A modo de ejemplo concreto, supóngase que la **capa oculta** aprende vectores de palabras de 300 dimensiones. En ese caso, los pesos de dicha capa quedan representados por una matriz de 10.000×300 : una fila por cada palabra del vocabulario y una columna por cada neurona oculta. Este es precisamente el tamaño que empleó Google en el trabajo original en el que presentó Word2Vec [8].

Al final, el objetivo de este modelo es aprender la matriz de pesos de la capa oculta. Una vez se tiene esta matriz, la capa de salida es inmediata.

Llegados a este punto, uno puede preguntarse cuál es la utilidad de representar las palabras mediante vectores de todo ceros salvo una única posición con valor 1. La respuesta reside en la eficiencia, multiplicar un vector *one-hot* de 1×10.000 por la matriz de 10.000×300 selecciona de forma inmediata la fila correspondiente a la palabra de entrada, sin necesidad de realizar el producto matricial completo.

Con esto se concluye que la capa oculta del modelo actúa en realidad como una tabla *lookup*: la salida de dicha capa es directamente el vector de representación de la palabra de entrada. Así, el vector de 1×300 resultante se pasa a la **capa de salida**, que es un clasificador de regresión *softmax*.

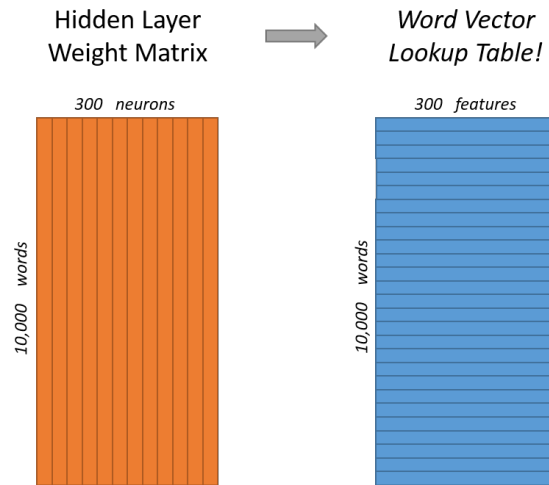


Figura 8.6: Matriz de pesos de la capa oculta del modelo Skip-gram: 10.000 filas (una por palabra del vocabulario) y 300 columnas (una por neurona oculta). Cada fila constituye el vector de representación de su palabra correspondiente. Fuente: [15].

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = \begin{bmatrix} 10 & 12 & 19 \end{bmatrix}$$

Figura 8.7: Multiplicación de un vector *one-hot* por la matriz de pesos de la capa oculta, obteniendo directamente el vector de representación de la palabra de entrada. Fuente: [15].

Cada neurona de salida produce un valor entre 0 y 1 aplicando la función $\exp(x)$ sobre el producto de su vector de pesos y el vector proveniente de la capa oculta. Para que todos los valores sumen 1 y formen una distribución de probabilidad, cada resultado se divide entre la suma de los valores de las 10.000 neuronas de salida [15].

Mejoras aplicadas al modelo básico Skip-gram

Retomando el ejemplo anterior, con 300 dimensiones y un vocabulario de 10.000 palabras, la red neuronal dispone de dos matrices de pesos: una correspondiente a la capa oculta y otra a la capa de salida. Cada una de ellas contiene un total de $300 \times 10.000 = 3$ millones de parámetros. Para ajustar correctamente este volumen de parámetros y evitar al mismo tiempo el sobreajuste se debe disponer de conjuntos de datos de entrenamiento de gran tamaño, lo que convierte el entrenamiento del modelo básico en un

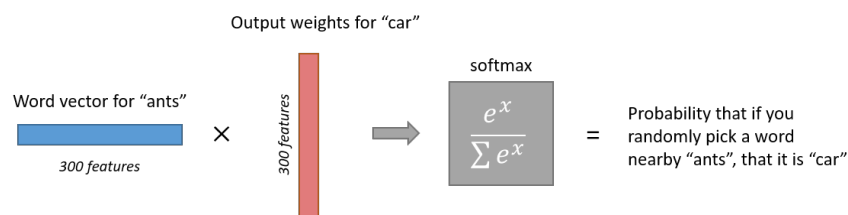


Figura 8.8: Cálculo de la salida de la neurona correspondiente a la palabra *car* en la capa *softmax* del modelo Skip-gram. Fuente: [15].

proceso computacionalmente costoso.

Para abordar este problema, Google publicó un segundo artículo [17] en el que se proponían dos innovaciones clave sobre el modelo básico:

- **Submuestreo de palabras frecuentes:** se reduce la frecuencia con la que las palabras muy comunes participan en el entrenamiento, disminuyendo así el número total de ejemplos procesados.
- **Negative Sampling:** se sustituye la función de optimización original (descenso del gradiente) por una alternativa más eficiente que, en lugar de actualizar todos los pesos de la red en cada iteración, únicamente modifica un subconjunto reducido de ellos.

Estas dos mejoras demostraron que, lejos de comprometer la calidad del modelo, se conseguía una reducción del coste computacional además de una mejora de la calidad de los vectores de palabras resultantes. Dado que en la implementación práctica de este trabajo se ha empleado el *Negative Sampling* como función de optimización, su funcionamiento se describe en detalle en la sección siguiente.

8.2. Función de optimización

Para comprender la mejora que introduce el *Negative Sampling*, es necesario partir de la formulación matemática original del modelo básico de Skip-gram y, a partir de ella, entender qué limitaciones motivan la adopción de esta nueva función de optimización [30].

El objetivo principal de Skip-gram es maximizar la probabilidad logarítmica promedio sobre todo el corpus de entrenamiento, lo que queda expresado mediante la siguiente fórmula:

$$\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-c \leq j \leq c \\ j \neq 0}} \log p(w_{t+j} | w_t)$$

Donde:

- c es el tamaño de la ventana de contexto. Valores más altos de c generan más ejemplos de entrenamiento y, por tanto, ofrecen mayor precisión, aunque a costa de un tiempo de entrenamiento superior.
- T es el número total de palabras del corpus de entrenamiento.
- w_t es la palabra de entrenamiento en la posición t .

La probabilidad condicional $p(w_{t+j} \mid w_t)$ del modelo básico de Skip-gram se define mediante la función *softmax* de la siguiente forma [30]:

$$p(w_O \mid w_I) = \frac{\exp(v'_{w_O}{}^\top v_{w_I})}{\sum_{w=1}^W \exp(v'_w{}^\top v_{w_I})}$$

Donde v_w y v'_w son, respectivamente, los vectores de representación de *entrada* y de *salida* de la palabra w , y W es el número total de palabras del vocabulario.

Esta formulación presenta un problema de escalabilidad: el coste de calcular $\nabla \log p(w_O \mid w_I)$ es proporcional a W , cuyo valor suele ser muy elevado (entre 10^5 y 10^7 términos). Ello implica que, en el denominador del *softmax*, es necesario acumular los valores exponenciales de todas las palabras del vocabulario en cada paso de entrenamiento, lo que hace que el proceso resulte prohibitivamente lento cuando el vocabulario es de gran escala. Es precisamente este coste el que motiva la introducción del *Negative Sampling* como función de optimización alternativa.

Esta nueva función de optimización persigue maximizar la similitud entre las palabras que aparecen en el mismo contexto y minimizarla cuando proceden de contextos distintos. Sin embargo, en lugar de aplicar esa minimización sobre todas las palabras del corpus, selecciona aleatoriamente k palabras, denominadas *muestras negativas*, y las emplea para optimizar la función objetivo [13]. La expresión de dicha función es la siguiente:

$$\log \sigma(v'_{w_O}{}^\top v_{w_I}) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma(-v'_{w_i}{}^\top v_{w_I}) \right]$$

Donde σ es la función sigmoide y $P_n(w)$ es la distribución de ruido a partir de la cual se extraen las muestras negativas. Esta distribución se calcula como la distribución unigrama de las palabras elevada a la potencia $\frac{3}{4}$, exponente elegido empíricamente en el trabajo original donde se introduce este algoritmo [17]:

$$P_n(w) = \frac{U(w)^{\frac{3}{4}}}{Z}$$

Donde $U(w)$ es la frecuencia de aparición de la palabra w en el corpus y Z es la constante de normalización que garantiza que la distribución sume 1.

Maximizar esta expresión conlleva maximizar el producto escalar en su primer término y minimizarlo en el segundo. En otras palabras, las palabras que aparecen en el mismo contexto se ven forzadas a adquirir representaciones vectoriales más similares entre sí, mientras que las que pertenecen a contextos distintos se ven forzadas a tener vectores menos similares [13].

En cuanto a la elección del hiperparámetro k , el segundo artículo de Google sobre Word2Vec [17] indica que un valor en el rango de 5 a 20 es adecuado para corpus de entrenamiento de tamaño reducido, mientras que para corpus de gran escala se recomienda utilizar valores de entre 2 y 5.

Capítulo 9

Implementación Cuántica: Algoritmo QWord2Vec

En el capítulo anterior se describió en detalle el algoritmo Word2Vec clásico y su variante Skip-gram, que emplea redes neuronales superficiales para aprender representaciones vectoriales de palabras a partir de grandes corpus de texto. En este capítulo se presenta *QWord2Vec*, una adaptación cuántica de dicho modelo propuesta por Ohno [18], cuyo objetivo es explorar si el uso de circuitos cuánticos paramétricos puede ofrecer ventajas en la calidad de las representaciones vectoriales de palabras.

En este capítulo se explorarán la arquitectura del circuito cuántico, la función de coste empleada para su optimización y la estrategia híbrida clásico-cuántica utilizada durante el entrenamiento.

9.1. Adaptación del algoritmo al entorno cuántico

El circuito cuántico de *QWord2Vec* sigue la misma estructura de Word2Vec, compuesta por una parte de **codificación** y una parte de **decodificación**. Está formado por puertas de rotación de Pauli (unitarias parametrizadas) y puertas de Pauli-X controladas para introducir entrelazamiento, organizadas en una estructura de capas con profundidad de circuito ajustable. Dado un vector *one-hot* como entrada para una palabra y un vector de salida que representa su contexto [18].

9.2. Diseño del Circuito Cuántico Parametrizado

Como se indicó con anterioridad, la arquitectura de *QWord2Vec* mantiene una estructura análoga a la del algoritmo clásico, dividiéndose en una fase de codificación y otra de decodificación. Esta similitud estructural se ilustra en la Figura 9.1.

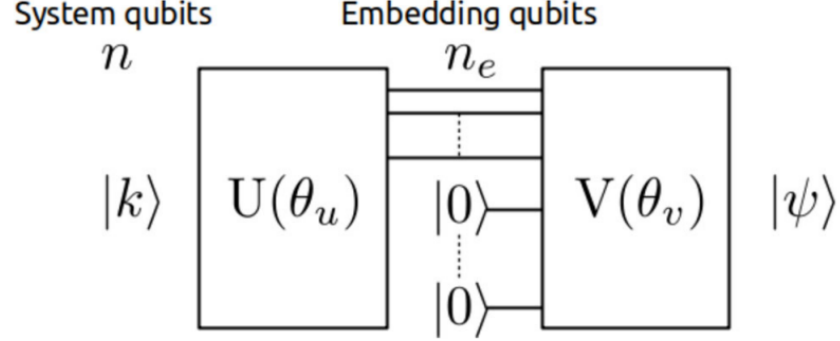


Figura 9.1: Estructura de QWord2Vec. Fuente: [18].

En la imagen anterior, se aprecia como hay dos matrices parametrizadas, $U(\theta_u)$ (matriz unitaria parametrizada) y $V(\theta_v)$ (matriz entrelazada unitaria). En este sistema, n denota el número total de qubits, mientras que n_e indica el número de qubits destinados específicamente a la representación de los *embeddings*, debiendo cumplirse la restricción $1 \leq n_e \leq n$.

La entrada al circuito viene dada por el estado cuántico $|k\rangle$, el cual representa a una palabra codificada mediante un vector *one-hot* tal que $0 \leq k \leq 2^n - 1$. A partir de este estado inicial, el sistema evoluciona hacia un estado de salida $|\psi\rangle$, estableciendo un mapeo directo de la misma manera al que realiza la red neuronal de Word2Vec. Los estados cuánticos de salida resultantes codifican la información correspondiente al contexto de la palabra de entrada, cuya extracción se lleva a cabo realizando mediciones en la base computacional (observable Pauli Z).

El modelo $U(\theta_u)$ y $V(\theta_v)$ utilizan las matrices unitarias parametrizadas y las matrices unitarias (U_{enta}) para proporcionar el entrelazamiento cuántico de la siguiente manera:

$$U(\theta) = \prod_{l=1}^L U_{\text{enta}} R_Z(\theta_z^{L-l+1})^{\otimes n} R_Y(\theta_y^{L-l+1})^{\otimes n} R_X(\theta_x^{L-l+1})^{\otimes n}$$

donde L representa la longitud del circuito cuántico. Por simplicidad, se asumirá que tanto $U(\theta_u)$ como $V(\theta_v)$ poseen la misma profundidad, a menos que se indique explícitamente lo contrario.

Para implementar U_{enta} , se ha usado puertas controladas Pauli X (CNOT), organizadas siguiendo una configuración de circuito por bloques [26]. Esta disposición favorece notablemente la expresividad del circuito y maximiza su capacidad para generar entrelazamiento. La Figura 9.2 muestra visualmente la estructura del circuito resultante al combinar las rotaciones de Pauli con el bloque de entrelazamiento descrito.

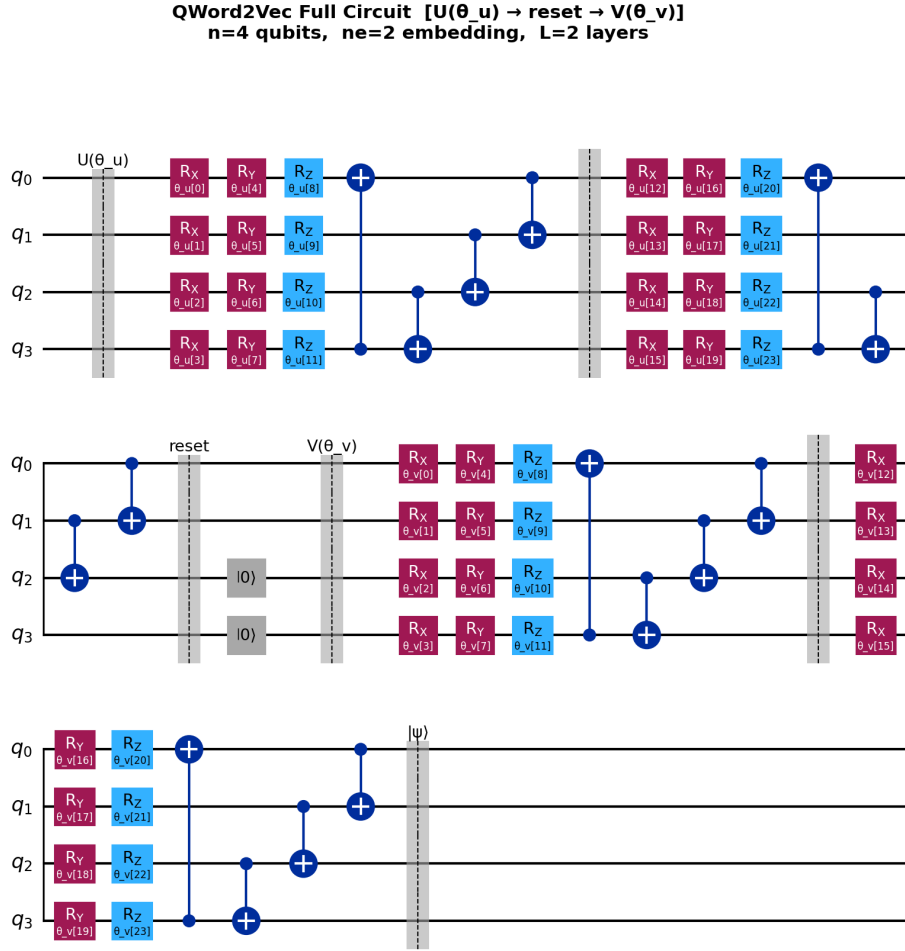


Figura 9.2: Circuito cuántico parametrizado de QWord2Vec con puertas de rotación de Pauli (R_X , R_Y , R_Z) y entrelazamiento con configuración por bloques mediante puertas CNOT. Elaboración propia.

El uso combinado de las tres puertas de rotación (R_X, R_Y, R_Z) permite que los estados cuánticos que representan a las distintas palabras alcancen una mayor dispersión en el espacio de Hilbert, superando las limitaciones de representatividad que se obtendrían empleando una sola matriz de rotación, tal y como se observa en la Figura 9.3.

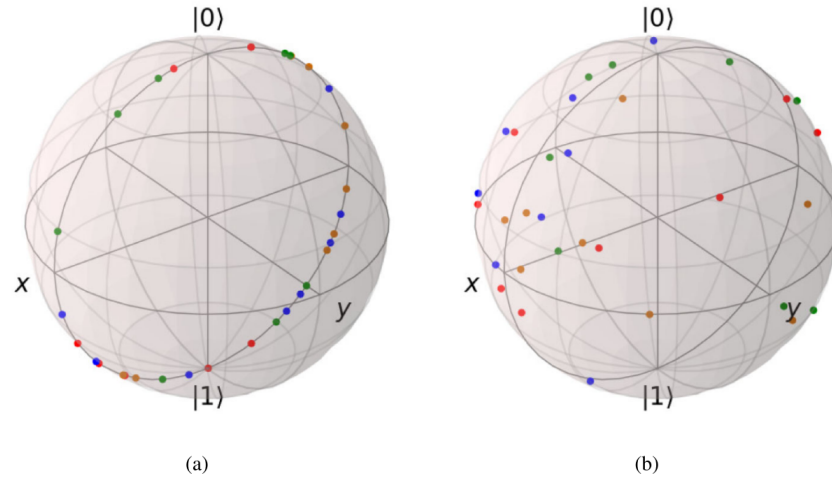


Figura 9.3: Representación de las palabras sobre la esfera de Bloch según las puertas de rotación empleadas en $U(\theta_u)$: (a) usando únicamente R_Y , y (b) usando R_X , R_Y y R_Z combinadas. Los puntos coloreados corresponden a las palabras y el número total de puntos es 32. El uso de las tres puertas de rotación de Pauli permite una mayor dispersión de las representaciones a lo largo del espacio de Hilbert. Fuente: [18].

9.3. Cálculo de similitud en hardware cuántico (Test de Swap / Fidelidad)

9.4. Definición de la función de coste cuántica

9.5. Estrategia de optimización híbrida (Clásica-Cuántica)

Capítulo 10

Experimentación y Análisis de Resultados

10.1. Metodología experimental

Para validar el desempeño de la propuesta QWord2Vec, se diseñó un conjunto de experimentos comparativos frente al Word2Vec clásico.

10.1.1. Dataset y preprocesamiento (corpus)

10.1.2. Entorno de simulación

10.2. Entrenamiento

10.2.1. Gráficas de pérdida: Word2Vec vs QWord2Vec

10.3. Evaluación de calidad de los embeddings

10.3.1. Pruebas de analogía y similitud

10.4. Comparativa de recursos computacionales y tiempos

Bibliografía

- [1] Felipe Almeida y Geraldo Xexéo. “Word Embeddings: A Survey”. En: *arXiv preprint* (2023). arXiv: 1901.09069 [cs.CL]. URL: <https://arxiv.org/abs/1901.09069>.
- [2] Frank Arute y col. “Quantum supremacy using a programmable superconducting processor”. En: *Nature* 574 (2019), págs. 505-510.
- [3] David Castaño Bandera. 4.2. *El estado de un qubit y la esfera de Bloch*. Universidad de Málaga (SCBI). 28 de mayo de 2025. URL: https://www.scbi.uma.es/web/wp-content/uploads/Jupyterbook/CICC_UMA/Teoria/html/docs/Part_03/Chapter_004_02/Section_002_el_estado_de_un_qubit_y_la_esfera_de_bloch_myst.html (visitado 16-02-2026).
- [4] El Español. *¿Cuántas palabras tiene el español?* El Español. 2021. URL: https://www.elespanol.com/curiosidades/lenguaje/cuantas-palabras-tiene-espanol-castellano/641935856_0.html (visitado 25-02-2026).
- [5] Ferrovial. *¿Qué son los números complejos?* Ferrovial. 2026. URL: <https://www.ferrovial.com/es/stem/numeros-complejos/> (visitado 16-02-2026).
- [6] Noelia Freire. *El número e: de Euler a la vida cotidiana*. National Geographic España. 18 de ene. de 2024. URL: https://www.nationalgeographic.com.es/ciencia/numero-e-euler-a-vida-cotidiana_21407 (visitado 16-02-2026).
- [7] GeeksforGeeks. *Phases of Natural Language Processing (NLP)*. GeeksforGeeks. 23 de jul. de 2025. URL: <https://www.geeksforgeeks.org/machine-learning/phases-of-natural-language-processing-nlp/> (visitado 16-02-2026).
- [8] Google. *word2vec*. Google Code Archive. 2013. URL: <https://code.google.com/archive/p/word2vec/> (visitado 26-02-2026).

- [9] Gopalan College of Engineering and Management. *Module 3: Quantum Computing & Quantum Gates*. Material de estudio. Applied Physics for Computer Science Engineering Stream. Gopalan College of Engineering y Management, 2022. URL: <https://www.gopalancolleges.com/gcem/pdf/syllabus/physics/cse/module-3-quantum-computing-quantum-gates.pdf> (visitado 16-02-2026).
- [10] Raffaele Guarasci, Giuseppe De Pietro y Massimo Esposito. “Quantum Natural Language Processing: Challenges and Opportunities”. En: *Applied Sciences* 12.11 (2022), pág. 5651. DOI: 10.3390/app12115651. URL: <https://www.mdpi.com/2076-3417/12/11/5651>.
- [11] Kevin Henner. *An intuitive introduction to text embeddings*. Stack Overflow. 9 de nov. de 2023. URL: <https://stackoverflow.blog/2023/11/09/an-intuitive-introduction-to-text-embeddings/> (visitado 23-02-2026).
- [12] IONOS. *Qubits: la base de los ordenadores cuánticos*. IONOS Digital Guide. 22 de feb. de 2023. URL: <https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/qubits/> (visitado 16-02-2026).
- [13] Akbar Karimi. *NLP’s word2vec: Negative Sampling Explained*. Baeldung Computer Science. 26 de mar. de 2025. URL: <https://www.baeldung.com/cs/nlps-word2vec-negative-sampling> (visitado 03-03-2026).
- [14] Inna Logunova y Olga Bolgurtseva. *Word2Vec: Why Do We Need Word Representations?* Serokell. URL: <https://serokell.io/blog/word2vec> (visitado 26-02-2026).
- [15] Chris McCormick. *Word2Vec Tutorial - The Skip-Gram Model*. McCormick ML. 19 de abr. de 2016. URL: <https://mccormickml.com/2016/04/19/word2vec-tutorial-the-skip-gram-model/> (visitado 26-02-2026).
- [16] Chris McCormick. *Word2Vec Tutorial Part 2 - Negative Sampling*. McCormick ML. 11 de ene. de 2017. URL: <https://mccormickml.com/2017/01/11/word2vec-tutorial-part-2-negative-sampling/> (visitado 26-02-2026).
- [17] Tomas Mikolov y col. “Distributed Representations of Words and Phrases and their Compositionality”. En: *arXiv preprint* (2013). arXiv: 1310.4546 [cs.CL]. URL: <https://arxiv.org/abs/1310.4546>.
- [18] Hiroshi Ohno. “Quantum circuits for word embeddings”. En: *Quantum Machine Intelligence* (2025). DOI: 10.1007/s42484-025-00309-w. URL: <https://doi.org/10.1007/s42484-025-00309-w>.

- [19] PennyLane. *Variational circuits — PennyLane*. PennyLane. URL: https://pennylane.ai/qml/glossary/variational_circuit (visitado 21-02-2026).
- [20] Rod Pierce. *Bra-Ket Notation*. MathsIsFun. 2025. URL: <https://www.mathsisfun.com/physics/bra-ket-notation.html> (visitado 16-02-2026).
- [21] Rod Pierce. *Fórmula de Euler para Números Complejos*. Disfruta Las Matemáticas. 2024. URL: <https://www.disfrutalasmatematicas.com/algebra/formula-euler.html> (visitado 16-02-2026).
- [22] Spencer Porter. *Understanding Cosine Similarity and Word Embeddings*. Medium. 28 de ago. de 2023. URL: <https://spencerporter2.medium.com/understanding-cosine-similarity-and-word-embeddings-dbf19362a3c> (visitado 25-02-2026).
- [23] Sangaku S.L. *Representación de números complejos en el plano*. Sangaku Maths. 2026. URL: <https://www.sangakoo.com/es/temas/representacion-de-numeros-complejos-en-el-plano> (visitado 16-02-2026).
- [24] Freya Shah. *Quantum Encoding: An Overview*. Quantum Zeitgeist. 2021. URL: <https://quantumzeitgeist.com/quantum-encoding-an-overview/> (visitado 18-02-2026).
- [25] Sam Sholtis. *Here's a better way to measure atomic qubits*. Futurity. 26 de mar. de 2019. URL: <https://www.futurity.org/quantum-computing-2018412/> (visitado 16-02-2026).
- [26] Sukin Sim, Peter D Johnson y Alán Aspuru-Guzik. “Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms”. En: *Advanced Quantum Technologies* 2.12 (2019), pág. 1900070.
- [27] SpinQ Technology. *Ultimate Guide to Quantum Gates: Everything You Need to Know*. 14 de mar. de 2025. URL: <https://www.spinquanta.com/news-detail/quantum-gates> (visitado 16-02-2026).
- [28] Cole Stryker y Jim Holdsworth. *What is NLP (Natural Language Processing)?* IBM. URL: <https://www.ibm.com/think/topics/natural-language-processing> (visitado 16-02-2026).
- [29] The MITRE corpustion. *The Bloch Sphere*. Intro to Quantum Software Development. 1 de jul. de 2022. URL: <https://stem.mitre.org/quantum/quantum-concepts/bloch-sphere.html> (visitado 16-02-2026).

- [30] Sik-Ho Tsang. *Review: Distributed Representations of Words and Phrases and their Compositionality — Negative Sampling (NLP)*. Medium. URL: <https://sh-tsang.medium.com/review-distributed-representations-of-words-and-phrases-and-their-compositionality-negative-ab9ebfc3f041> (visitado 01-03-2026).
- [31] Charles M. Varmantchaonala y col. “Quantum Natural Language Processing: A Comprehensive Survey”. En: *IEEE Access* 12 (2024), págs. 99578-99598. DOI: 10.1109/ACCESS.2024.3420707.
- [32] Dominic Widdows y col. “Quantum Natural Language Processing”. En: *KI - Künstliche Intelligenz* 38.4 (dic. de 2024), págs. 293-310. DOI: 10.1007/s13218-024-00861-w. URL: <https://doi.org/10.1007/s13218-024-00861-w>.
- [33] Peter Wittek y col. “Combining Word Semantics within Complex Hilbert Space for Information Retrieval”. En: *Quantum Interaction*. Vol. 8369. Lecture Notes in Computer Science. Springer, 2014, págs. 160-171. URL: <https://www.diva-portal.org/smash/get/diva2:887708/FULLTEXT01.pdf>.
- [34] Yanying Wu y Quanlong Wang. *A Categorical Compositional Distributional Modelling for the Language of Life*. 2019. arXiv: 1902.09303 [q-bio.QM]. URL: <https://arxiv.org/abs/1902.09303>.
- [35] Han-Sen Zhong y col. “Quantum computational advantage using photons”. En: *Science* 370 (2020), págs. 1460-1463.

Glosario

CBOW Continuous Bag of Words.	IA Inteligencia Artificial.
CQM Categorical Quantum Mechanics.	LLM Large Language Model.
CVC Circuitos Variacionales Cuánticos.	ML Machine Learning.
DisCoCat Distributional Compositional Categorical.	MNL Modelo Neuronal del Lenguaje.
DL Deep Learning.	NER Named Entity Recognition.
DPT Dependency Parse Tree.	NISQ Noisy Intermediate-Scale Quantum.
DVS Descomposición en Valores Singulares.	NLP Natural Language Processing.
GPT Generative Pre-trained Transformer.	PLN Procesamiento del Lenguaje Natural.
	PLNC Procesamiento del Lenguaje Natural Cuántico.
	POS Part-of-Speech.
	QML Quantum Machine Learning.
	Qubit Quantum Bit.
	WSD Word Sense Disambiguation.